

PDM Fuzzy Risk Model — Technical Documentation

Table of Contents

Purpose of the model	2
01 — How the risk index (mrisk index) is calculated.....	3
1. Where the score comes from: the three live inputs	3
2. The five stages of the pipeline	3
3. Why a fuzzy model?	8
4. Known limits and roadmap	8
02 — Configuration parameters reference (config.py).....	9
1. File paths.....	9
2. Weather data format.....	9
3. Hemisphere and seasonal settings.....	10
4. Phenological window variability	10
5. Fuzzy model parameters	10
6. Risk classification thresholds.....	11
7. Streak penalty	11
8. Growing Degree Days.....	12
9. Leaf-wetness estimation.....	12
10. Report settings.....	13
03 — JSON schema (pest.json)	14
1. Field-by-field reference.....	14
2. Validation rules and defaults	17
04 — Climate criteria and disease selection	19
1. Which climate variables the model uses	19
2. How each variable enters the fuzzy decision.....	19
3. From climate criteria to disease selection	20
4. Climate issues.....	21
05 — Database population guide.....	22
1. Literature review.....	22
2. Define the pest_key	22
3. Edit pest.json.....	23
4. Edit db_crops_PDM.xlsx.....	24
5. Update config.py	25
6. Run and validate	25
7. Common failure modes and recovery.....	26

Purpose of the model

The PDM (Pest & Disease Model) is a **multi-rule fuzzy inference system** that, given a daily weather series (T_max, T_min, relative humidity, rainfall), returns a risk index (0–100) and a class (*Low-Moderate-High-Critical-Out of season*) for every combination of (*day, crop, pest*). Risk rules are declared in an Excel worksheet; biological parameters (thermal thresholds, phenological window, wetness requirements) are declared in a JSON file. The scoring engine is written entirely in Python and pursues three operational goals:

1. **Alert** the user when weather conditions become favourable for the establishment or outbreak of a pest;
2. **Schedule** treatment windows on the basis of host phenology (BBCH stages) and pest epidemiology;
3. **Make the decision reproducible**: each score is traceable back to the rules that fired (see the *detail* column of the CSV output).

The approach is multi-crop and is designed to be extended without touching the code: it is enough to update the two data files (.xlsx and .json).

01 — How the risk index (mrisk index) is calculated

The PDM is a **multi-rule Mamdani fuzzy system** enriched with:

- a **fuzzy phenological window** that modulates the score when the host is not in a susceptible stage;
- a **lethal-temperature guard** that suppresses risk when T is incompatible with the organism's survival;
- a **Mills-type leaf-wetness gate** for pathogens with an obligate infection requirement;
- a **streak penalty** for pathogens that require persistent humidity over consecutive days.

The whole computation is wrapped inside the `score_day()` function of `risk_model.py` and is sketched below.

1. Where the score comes from: the three live inputs

For **each day** of the weather series and **each pair of crop-pest** the model combines three data sources:

1. **Declarative fuzzy rules** in `db_crops_PDM.xlsx`. Every row is a rule of the form “if $RH \in [a,b]$ and $T \in [c,d]$ and $rainfall \geq R_{min}$, then $risk = Critical/High/Moderate/Low$ ”. A real example (*Plasmopora viticola* on grapevine):

Humidity	Temperature	Rainfall	Risk
>90%	$8 \leq T \leq 25$	≥ 10 mm	Critical
>60%	$8 \leq T \leq 20$	≥ 10 mm	High
>60%	$5 \leq T < 8$	≥ 5 mm	Moderate
>60%	$T > 30$	≥ 10 mm	Low

2. **Biological parameters** in `pest.json`. For each pest, they define the organism's “personality”: GDD base temperature (t_{base}), lethal temperature ($t_{lethal_max/min}$), phenological window ($phenology_window_gdd$, $pheno_fraction$), minimum number of consecutive wet days (min_streak), optional leaf-wetness thresholds ($min_wetness_hours_critical/high$).
3. **Daily weather series** in `weather_data.xlsx`. Required columns: `date`, `temp_max`, `temp_min`, `humidity`, `rainfall`. The loader accepts any column order, auto-detects the date format and interpolates the temperature when missing. If the code works in app, the `.xlsx` file is replaced with the API call to daily weather data, forecasts, and the required history.

The **join keys** are the `crop` (e.g. *Vineyard*) and the `pest_key` (e.g. *Plasmopora viticola*); they must match **exactly** between the Excel worksheet and the JSON file. This is the golden rule when populating the database.

2. The five stages of the pipeline

STEP 1 — Data loading (`Load_*` functions)

`Load_rules()`, `Load_pest_config()` and `Load_weather()` read the three files and normalise the textual fields. In particular:

- Numerical ranges in the Excel file are written in a human-readable format ($60-80\%$, $8 \leq T \leq 25$, $\geq 10 \text{ mm}$, *whatever*). Internal parsers (`_parse_humidity`, `_parse_temp`, `_parse_rainfall`) turn them into (*lo*, *hi*) numerical pairs. The special string *whatever* means “don’t care” and maps to (*0*, *100*) for RH and (*-999*, *999*) for T.

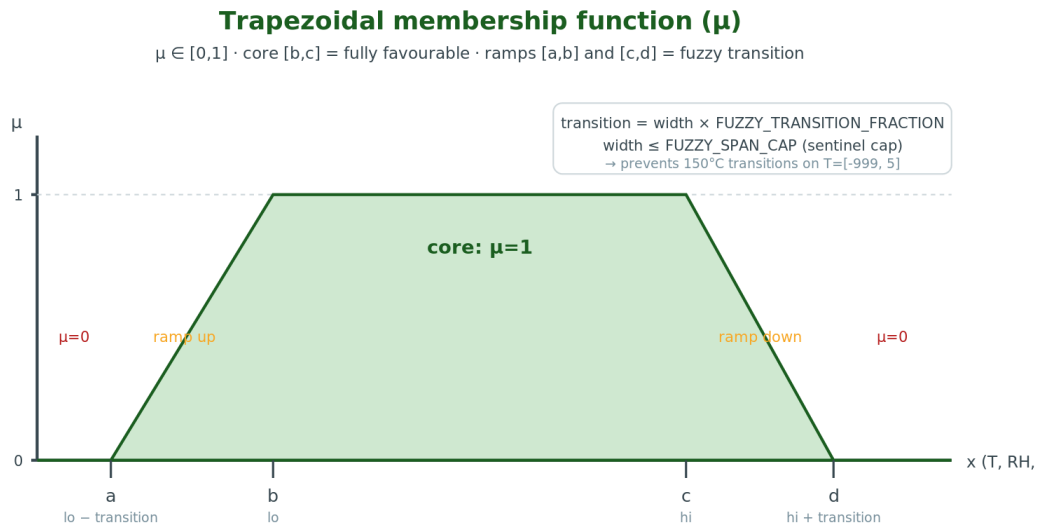
STEP 2 — Feature engineering (`compute_features`)

Around 30 derived columns are added to the raw weather table. The most important are:

- `temp_avg` = $(T_{max} + T_{min}) / 2$
- Rolling averages** over 3, 7, 14 days of T, RH and cumulative rainfall. These are used for pathogens that respond slowly to the environment.
- Daily leaf-wetness hours** (`wetness_h`): combined contribution of high RH and rainfall after Pedro & Gillespie (1982) and Sentelhas et al. (2008). Capped at 24 h/day.
- Cumulative GDD (Growing Degree Days)**: computed for **every** `t_base` present in `pest.json` (the set $\{0, 4, 5, 8, 9, 10, 12, 13, 14, 15, 18\}$ °C). Each pest then picks its own GDD series. The cumulator resets at `GDD_RESET_MONTH` from `config.py` (1 for the northern hemisphere, 7 for the southern hemisphere).
- Humidity streaks**: counters of consecutive days with $RH \geq 60, 70, 80, 85, 90, 95 \%$. Used by the penalty in step 6.
- VPD** (Vapour Pressure Deficit) via the Tetens equation — precomputed for downstream analysis, not yet wired into the score.

STEP 3 — Fuzzy membership functions

Every weather quantity is compared with the rule’s bounds via a **trapezoid**:



The transition width is proportional to the range width ($width \times FUZZY_TRANSITION_FRACTION$, default 15%), with two safeguards:

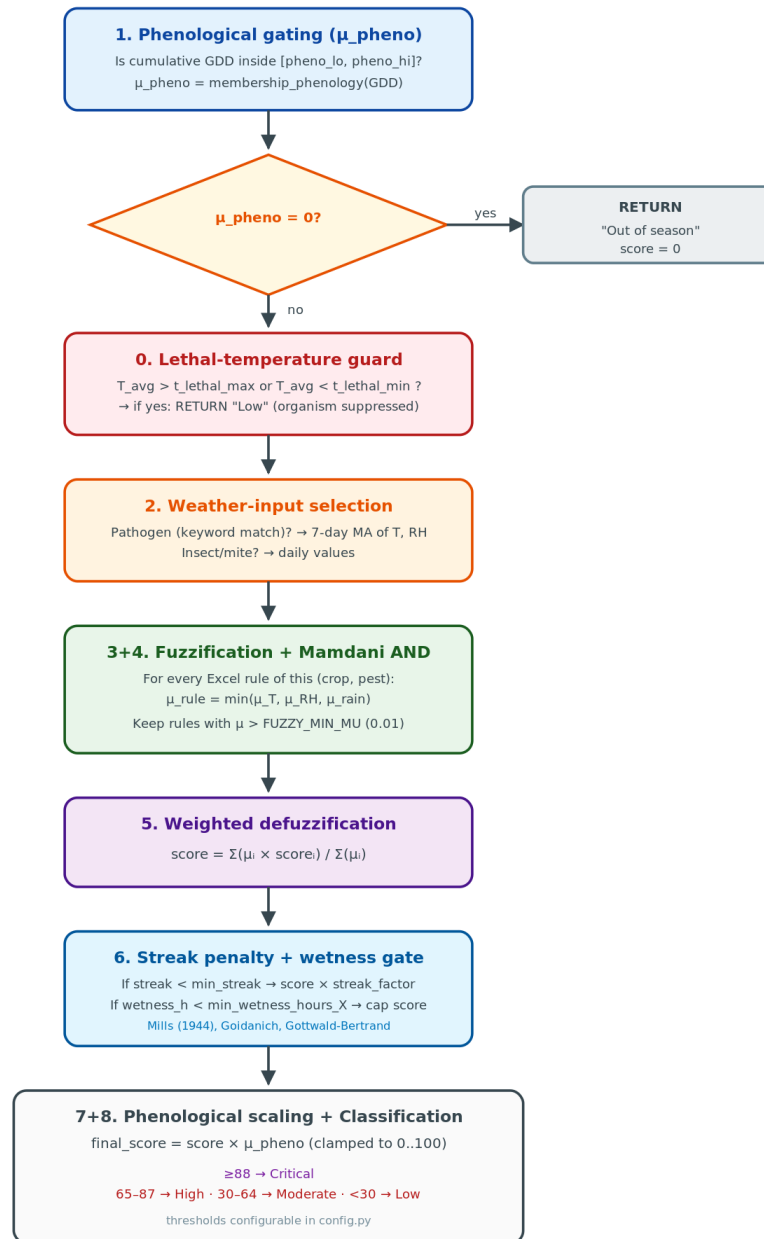
- a **minimum width** (2°C for T, 5% for RH) to avoid overly sharp edges;
- a **maximum width** `FUZZY_SPAN_CAP` = 30. Without the cap a sentinel rule such as $T = [-999, 5]$ (meaning “ $T < 5$ ”) would generate 150°C-wide transitions and would fire spuriously even at 20°C. With the cap the transition remains ≤ 30 units.

For rainfall the trapezoid is asymmetric: $\mu = 1$ on $[rain_min, 3 \cdot rain_min]$, then decreases to zero at $6 \cdot rain_min$ to model the inhibition of spore dispersal under heavy waterlogging.

STEP 4 — `score_day()`: the heart of the model

`score_day()` — decision flow for one (day, crop, pest) triplet

Each box maps directly to a section of `score_day()` in `risk_model.py`



4.1 — Phenological gating (μ_{pheno})

A plant that is not in a susceptible stage carries no risk, regardless of the weather. For each pest the model computes the phenological position of the host in GDD units:

GDD_{now} = cumulative GDD from 1 January (or from GDD_RESET_MONTH) using the pest's pheno_t_base (defaults to t_base)

It then compares *GDD_{now}* with the window declared in *pest.json*. Two modes are supported:

1. **Absolute window** — *phenology_window_gdd*: [80, 1200] (in the pest's GDD units);
2. **Normalised window** — *pheno_fraction*: [0.036, 0.545] (fraction of the annual reference GDD). This is the preferred mode because it works across any climate without re-calibration.

The phenological membership is **not binary** but trapezoidal. A soft transition ($PHENO_FUZZY_MARGIN_FRAC \times width$, default 12%) at the edges absorbs the inter-annual variability of plant development.

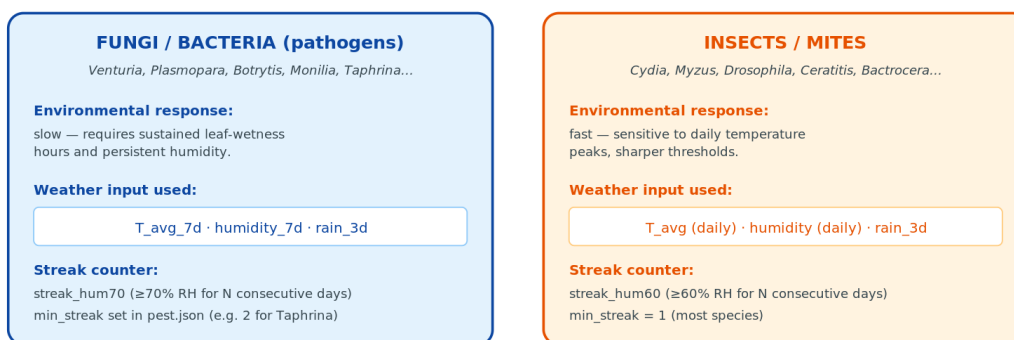
When $\mu_{pheno} = 0$ the function **exits immediately** with $risk_class = "Out\ of\ season"$ and score 0: no CPU cycles are wasted on fuzzification.

4.2 — Lethal-temperature guard

A double safeguard: if $temp_avg > t_lethal_max$ (organism cooked) or $temp_avg < t_lethal_min$ (organism frozen), the model returns *Low* with score 0 and a diagnostic message. This captures observed realities that are hard to express with fuzzy rules alone (e.g. Taphrina “switches off” above 30°C).

4.3 — Automatic weather-window selection

Fungi and bacteria respond slowly to the environment; insects and mites respond quickly.



A hard-coded list of **genus keywords** in *risk_model.py* (`_PATHOGEN_KEYWORDS`) classifies the pest as “pathogen” or “non-pathogen”:

- **Pathogen:** uses the **7-day moving average** of T and RH ($temp_avg_7d, humidity_7d$) — best captures the infection/sporulation cycles typical of fungi (Plasmopara, Venturia, Botrytis, Monilia...). The reference streak counter is *streak_hum70*.
- **Insect/mite:** uses **daily** values — best captures rapid responses to short thermal windows (Cydia, Myzus, Drosophila, Ceratitis...). The reference streak counter is *streak_hum60*.

Rainfall is always taken as the cumulative *rain_3d* (3 days), consistent with the idea that spore leaching or conidial release requires a sustained event. Explicit exception: pests declaring *rain_window_days: 1* in *pest.json* (e.g. Plasmopora following the Goidanich rule “≥ 10 mm in 24 h”) use the raw daily rainfall.

4.4 — Fuzzification + Mamdani aggregation

For every Excel rule belonging to this (*crop*, *pest*) the model computes:

```
 $\mu_T$  = membership_temp( $t\_eff$ , rule['temp_lo'], rule['temp_hi'])
 $\mu_{RH}$  = membership_humidity( $h\_eff$ , rule['hum_lo'], rule['hum_hi'])
 $\mu_{rain}$  = membership_rainfall( $rain\_eff$ , rule['rain_min'])
```

```
 $\mu_{rule}$  = min( $\mu_T$ ,  $\mu_{RH}$ ,  $\mu_{rain}$ ) ← Mamdani AND
```

Only rules with $\mu_{rule} > FUZZY_MIN_MU$ (default 0.01) are kept, in order to discard the noise of infinitesimal memberships.

4.5 — Weighted defuzzification

The surviving rules vote in proportion to their activation:

$$\text{score} = \Sigma (\mu_i \times \text{risk_score}_i) / \Sigma \mu_i$$

risk_score_i is the integer associated with the risk level declared in the rule: Critical → 100, High → 80, Moderate → 40, Low → 10. The result is a continuous score in the range [10, 100].

4.6 — Streak penalty + leaf-wetness gate

Two epidemiologically motivated corrections:

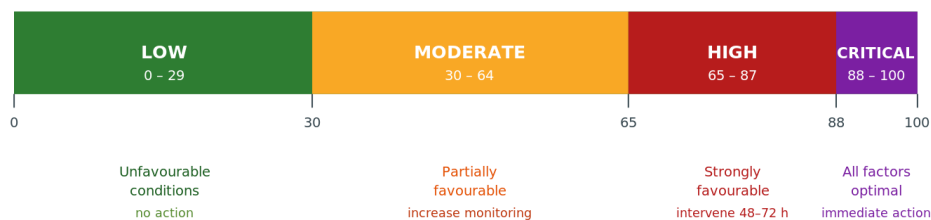
- If the consecutive days of high RH (*streak*) are below *min_streak*, the score is multiplied by a *streak_factor*. For pathogens with an obligate biological requirement (e.g. *Taphrina deformans* — needs two consecutive days with ≥ 15 h leaf wetness to infect), *min_streak* ≥ 2 enforces a **hard floor of 0** (an isolated favourable day scores zero). For the others (*min_streak* = 1) a soft floor *STREAK_MIN_FACTOR* = 0.30 is applied to avoid sharp cut-offs.
- When *min_wetness_hours_critical* or *min_wetness_hours_high* are declared in *pest.json* (e.g. *Venturia inaequalis*: 6 h for Critical) and the estimated *wetness_h* falls short, the score is **capped** just below the corresponding threshold. This applies the “Mills table” without rewriting the Excel rules.

4.7 — Final phenological scaling

$$\text{final_score} = \text{clamp}(0, 100, \text{score} \times \mu_{\text{pheno}})$$

When the host is **partially** susceptible ($\mu_{\text{pheno}} \in (0, 1)$) the score is scaled accordingly. A “truly Critical” day that falls in the phenological ramp-up zone may therefore surface as “High”.

4.8 — Classification



Thresholds live in *config.py*:

Class	Score	Operational meaning
Low	0–29	Unfavourable conditions, no action
Moderate	30–64	Partially favourable, increase monitoring
High	65–87	Strongly favourable, intervention within 48–72 h
Critical	88–100	All factors optimal, immediate action
Out of season	n/a	Host outside its phenological window

STEP 5 — Iteration (*run_model*)

A plain loop: for every (*crop*, *pest_key*) indexed in memory and for every row of the weather table, call *score_day()* and append the result. The final DataFrame holds one row per (*date*, *crop*, *pest*) with the score, the class, a weather snapshot for the day and the diagnostic *detail* string that reports the dominant rule (*best_rule*), its μ value and the values of *μ_{pheno}* , *streak*, *wetness_h*.

3. Why a fuzzy model?

The three classic alternatives are Boolean rules (if/else), statistical models (logistic regression), neural networks. For the PDM the fuzzy approach was chosen because:

Criterion	Mamdani fuzzy	If/else	Logistic	NN
Driven by expert knowledge (no training set required)	✓	✓	✗	✗
Robust to small weather errors ($\pm 1^\circ\text{C}$)	✓	✗	✓	✓
Explainable: every score traceable to firing rules	✓	✓	partial	✗
Adding a new pest = adding rows and JSON entries	✓	✓	needs re-fit	needs re-fit
Mills-style soft thresholds expressed naturally	✓	✗	partial	✓

In particular, fuzzy rules naturally absorb the noise of weather measurements: a day with RH 89% is not excluded by a rule “RH > 90%” but contributes with $\mu \approx 0.85$, which is consistent with the actual physiology of the pathogen.

4. Known limits and roadmap

- The model assumes a **homogeneous microclimate** on the site (one weather station per dataset). For hilly terrain or large vineyards we suggest partitioning the weather series per zone.
- Excel rules support **at most three dimensions** (T, RH, R). Additional variables — VPD, solar radiation, wind — are already pre-computed (*vpd*, *vpd_3d*, *vpd_7d*) but not yet wired into the score. The structure is ready to integrate them without breaking backward compatibility.
- *gdd_annual_ref* is derived from the available weather series: with fewer than 350 days it uses a linear projection, otherwise the mean across complete seasons. For a first deployment in a new site at least one full year of historical weather is recommended.
- The *_PATHOGEN_KEYWORDS* classifier is a hard-coded list: integrating new pathogen families may benefit from moving the flag into *pest.json* (e.g. a *pest_type*: “fungus” / “insect” field).

02 — Configuration parameters reference (*config.py*)

Every parameter that is meant to be touched by the user lives in *config.py*. This document explains, group by group, what each one does, the unit, the recommended range and the consequence of changing it.

The golden rule, repeated at the top of the file, applies: **do not modify the other source files**; all tuning belongs here.

1. File paths

```
RULES_EXCEL      = Path("db_crops_PDM.xlsx")
PEST_CONFIG_JSON = Path("pest.json")
WEATHER_EXCEL    = Path("weather_data.xlsx")
OUTPUT_DIR       = Path("output")
```

Paths are **relative** to the current working directory when *main.py* is launched. To run the model from another folder, either invoke it with *python /full/path/main.py* (and accept the implicit cwd) or replace the relative paths with absolute ones. The output directory is created automatically if it does not already exist.

2. Weather data format

```
WEATHER_COLUMNS = {
    "date": "date",
    "temp_max": "temp_max",
    "temp_min": "temp_min",
    "humidity": "humidity",
    "rainfall": "rainfall",
}
DATE_FORMAT = "auto"
```

Key	What it represents	Unit	Notes
<i>date</i>	Column name holding the date in the Excel file	—	Required
<i>temp_max</i>	Daily maximum temperature column	°C	Numeric; NaNs are linearly interpolated
<i>temp_min</i>	Daily minimum temperature column	°C	Numeric
<i>humidity</i>	Daily mean relative humidity column	%	Missing values forward-filled (default 70%)
<i>rainfall</i>	Daily rainfall column	mm	Negative values clipped to 0

If the weather provider uses different column names (e.g. *Tmax*, *RH_mean*), edit only the *values* on the right of the dictionary, keep the keys on the left untouched.

DATE_FORMAT accepts *"auto"* (default, auto-detected from the first non-null row) or one of *"%Y-%m-%d"*, *"%d-%m-%Y"*, *"%d/%m/%Y"*, *"%Y/%m/%d"*. Pin a fixed format only if the auto-detector returns wrong results for ambiguous inputs (e.g. *01/02/2026* may be January or February depending on locale).

3. Hemisphere and seasonal settings

`GDD_RESET_MONTH = 1`

Value	Meaning
1	Northern hemisphere — GDD cumulator resets on 1 January
7	Southern hemisphere — GDD cumulator resets on 1 July
1 (tropics)	Leave it at 1; phenological windows are irrelevant for tropical pests that use $[0, 9999]$

When `GDD_RESET_MONTH = 7`, the internal `season_year` label maps January–June days to the previous calendar year so the cumulative GDD spans the natural agronomic season. Any value other than 1 or 7 is accepted but rarely useful.

4. Phenological window variability

`PHENO_FUZZY_MARGIN_FRAC = 0.12`

Fraction of the window used as soft transition zone at each edge. `0.12` covers approximately ± 1.5 SD of the inter-annual phenological variability documented by Linkosalo et al. (2008) and Menzel et al. (2006) — roughly ± 50 – 100 GDD5, equivalent to ± 7 – 14 days of bud burst variability across years.

Recommended range	Effect
<code>0.05–0.10</code>	Tight window — best for pests where the susceptible stage is short and well-defined (e.g. flowering-only infections)
<code>0.10–0.15</code> (default)	Balanced — recommended starting point
<code>0.15–0.25</code>	Wide ramp — for crops with very variable vegetative cycles (banana, papaya, year-round tropical species)

Setting it to `0.0` turns the gate into a binary in/out check.

5. Fuzzy model parameters

`FUZZY_TRANSITION_FRACTION = 0.15`

`FUZZY_SPAN_CAP = 30.0`

`FUZZY_MIN_MU = 0.01`

Parameter	Unit	What it controls	Typical impact when changed
<code>FUZZY_TRANSITION_FRACTION</code>	fraction	Width of the ramp at each side of the trapezoid (as a fraction of the rule's width)	Larger → softer transitions, more rules fire near the edges; smaller → sharper, behaves more like Boolean rules
<code>FUZZY_SPAN_CAP</code>	°C or %RH	Maximum effective width used when	Should stay between 20 and 40; below 20

		computing the ramp. Prevents sentinel rules such as $T = [-999, 5]$ from spawning 150°C-wide ramps	the cap activates also for normal rules; above 40 sentinel rules become problematic
<i>FUZZY_MIN_MU</i>	dimensionless	Minimum μ value for a rule to be considered “fired”	Lower (0.005–0.01) → keeps very weak rules, more noise; higher (0.05–0.10) → reduces noise but can drop legitimate marginal cases

Operational hint. When validating a new pest, leave the fuzzy parameters at their defaults. Only adjust *FUZZY_TRANSITION_FRACTION* upwards (e.g. to 0.20) if your weather station is known to be noisy or far from the orchard.

6. Risk classification thresholds

RISK_THRESHOLD_CRITICAL = 88

RISK_THRESHOLD_HIGH = 65

RISK_THRESHOLD_MODERATE = 30

Class	Score	Operational meaning
Low	0–29	Unfavourable, no action
Moderate	30–64	Partially favourable, increase monitoring
High	65–87	Strongly favourable, intervention within 48–72 h
Critical	88–100	All factors optimal, immediate action

These integers are referenced in three places: the validation summary, the PDF report and the *score_day()* classification step. Lowering *RISK_THRESHOLD_CRITICAL* makes Critical alerts more frequent (higher recall, lower precision). Raising *RISK_THRESHOLD_HIGH* shrinks the High band toward Moderate. The thresholds must satisfy *MODERATE* < *HIGH* < *CRITICAL*, otherwise the classifier would never produce one of the classes.

7. Streak penalty

STREAK_MIN_FACTOR = 0.30

Soft floor applied when a pest with *min_streak* = 1 (mostly insects) sees the streak counter at 0. With *STREAK_MIN_FACTOR* = 0.30 an isolated favourable day still scores at least 30% of the defuzzified score, smoothing the cut-off. Pests with *min_streak* ≥ 2 (e.g. obligate-wetness pathogens like *Taphrina deformans*) bypass this floor and apply a strict linear ramp instead — the biological requirement is a hard one.

Value	Behaviour
0.0	Hard floor: no score until the streak threshold is met
0.20–0.40	Soft, default range
0.50+	Streak almost ignored — discouraged unless you want to disable the mechanism

8. Growing Degree Days

```
PHENOLOGY_T_BASE = 5.0
```

```
PEST_T_BASES = {0.0, 4.0, 5.0, 8.0, 9.0, 10.0, 12.0, 13.0, 14.0, 15.0, 18.0}
```

```
GDD_CHART_BASES = {  
    "Fungi (0 C)": 0.0,  
    "Pathogens (5 C)": 5.0,  
    "Insects (10 C)": 10.0,  
}
```

- **PHENOLOGY_T_BASE** — fallback *t_base* (°C) used when a pest's JSON entry does not specify one. Default 5°C is appropriate for most temperate tree crops.
 - **PEST_T_BASES** — list of all distinct *t_base* values in use across *pest.json*. *compute_features()* pre-computes one *gdd_cum_{X}b* column per value so that *score_day()* can look up the phenologically correct GDD for each pest. **When you add a new pest whose *t_base* is not yet in the set, update this list too**, otherwise the per-pest GDD column will not be created and the fallback will be *gdd_cum_pheno* (5°C base) — biologically incorrect for that pest.
 - **GDD_CHART_BASES** — the three GDD curves drawn in the PDF report. Purely cosmetic.
-

9. Leaf-wetness estimation

```
WETNESS_HOURS_BY_RH = [  
    (95, 20), # RH ≥ 95% → 20 h/day  
    (85, 14),  
    (75, 8),  
    (65, 4),  
    (0, 0),  
]
```

```
WETNESS_HOURS_BY_RAIN = [  
    (5.0, 6), # rainfall ≥ 5 mm → +6 h  
    (1.0, 4),  
    (0.1, 2),  
    (0.0, 0),  
]
```

These two lists implement the empirical leaf-wetness model derived from Pedro & Gillespie (1982) and Sentelhas et al. (2008). The two contributions are **added** and the total is **capped at 24 h/day** inside *_wetness_hours()*. Each list is read as a piecewise-constant lookup table; the entries must be sorted in any consistent order (the loader sorts internally on the first column).

The output column (*wetness_h*) is then summed over 3 days into *wetness_3d* and used inside the Mills-type gate when *min_wetness_hours_critical/high* are set in *pest.json*.

Calibration note. If a site has a working leaf-wetness sensor (rare in Europe, common in North-American IPM stations), the model can ingest it directly by adding the column to *WEATHER_COLUMNS* and replacing the call to *_wetness_hours()* in *compute_features()* with a passthrough.

10. Report settings

```
REPORT_TITLE      = "Pest & Disease Risk Report"
REPORT_SUBTITLE   = "Agrobit iAgro Platform – Multi-crop Fuzzy Risk Model v2.0"
REPORT_FILENAME   = "pest_risk_report.pdf"
```

```
REPORT_SECTIONS = {
    "cover":          True,
    "weather_overview": True,
    "gdd":             True,
    "risk_timeline":   True,
    "seasonal_heatmap": False,
    "distribution":     False,
}
```

Set any value to *False* to skip the corresponding page when generating the PDF. *seasonal_heatmap* and *distribution* are reserved for a future release — toggling them currently has no effect.

03 —JSON schema (*pest.json*)

Every entry in *pest.json* describes one (*crop*, *pest*) pair and supplies the data that the engine needs to gate, scale and clamp the fuzzy score.

The file is a two-level dictionary keyed first by **crop**, then by **pest_key**. The *pest_key* must match exactly the value used in *db_crops_PDM.xlsx* (extracted from the column *Pest / Pathogen* as everything before the first comma, plus an optional parenthetical variant).

```
{
  "Vineyard": {
    "Plasmopora viticola": { ... },
    "Botrytis cinerea":    { ... },
    ...
  },
  "Apple": {
    "Venturia inaequalis": { ... },
    "Cydia pomonella":     { ... }
  }
}
```

Below, every recognised field is documented. Fields are either **required**, **recommended** or **optional/documentary** — the loader (*load_pest_config()*) treats most of them as optional and applies sensible defaults.

1. Field-by-field reference

1.1 Identification

Field	Type	Status	Meaning
<i>label</i>	string	optional	Common name in English (e.g. “Apple scab”). Used only in human-facing reports, not in the engine.
<i>note</i>	string	optional	Free-text agronomic note attached to the entry. Reviewers can drop here decisions and exceptions.
<i>source</i>	string	recommended	Scientific references (papers, EPPO datasheets, IPM bulletins) that ground the parameter values. Crucial for traceability.
<i>model_reference</i>	string	optional	Name of the third-party model behind the parameters (e.g. “Maryblyt (Cougarblight)”, “Mills 1944”).
<i>mills_table_reference</i>	string	optional	Synthetic citation of the Mills-style table used to derive <i>min_wetness_hours_*</i> .
<i>goidanich_rule</i>	string	optional	One-liner of the Goidanich infection rule for downy mildew (<i>Plasmopara viticola</i>).

1.2 Thermal parameters

Field	Type	Status	Unit	Meaning
<i>t_base</i>	float	required	°C	Lower temperature threshold for GDD accumulation ($GDD = \max(0, T_{avg} - t_{base})$). Drives both <i>gdd_cum_{X}b</i> and, by default, the phenological gate.
<i>pheno_t_base</i>	float	optional	°C	Override of <i>t_base</i> for phenological gating only. Use this when the host phenology (e.g. pear flowering, GDD5) is decoupled from the pathogen activity (e.g. Erwinia 18°C threshold).
<i>t_optimal_min</i>	float	optional	°C	Lower bound of the optimum range. Documentary — <i>not</i> consumed by the engine, but useful in dashboards.
<i>t_optimal_max</i>	float	optional	°C	Upper bound of the optimum range. Documentary.
<i>t_lethal_min</i>	float	optional	°C	Hard guard: $T_{avg} < t_{lethal_min} \rightarrow$ score 0, class Low. Use sparingly (most pests have no rigid lower lethal limit on a daily mean).
<i>t_lethal_max</i>	float	recommended	°C	Hard guard: $T_{avg} > t_{lethal_max} \rightarrow$ score 0, class Low. Captures heat-shutdown effects (Taphrina > 30°C, Erwinia > 37°C).
<i>t_lethal_infection_max</i>	float	legacy	°C	Backward-compatible alias for <i>t_lethal_max</i> (Taphrina entries).
<i>t_upper_threshold</i>	float	optional	°C	Documentary upper limit (e.g. Cydia 31°C). Not consumed by the engine.

Operational hint. *t_base* should normally match the literature value used in the source models. When in doubt, use 5°C for fungi/bacteria, 10°C for most insects.

1.3 Phenological window

The engine supports **two equivalent ways** to declare the host susceptibility window. The normalised form takes precedence if both are present.

Field	Type	Status	Unit	Meaning
<i>phenology_window_gdd</i>	[float, float]	optional	GDD units of <i>pheno_t_base</i>	Absolute window. Easy to set against published data, but climate-dependent.
<i>pheno_fraction</i>	[float,	recommended	fraction of	Climate-

	<i>float</i>		annual GDD	independent window expressed as fractions of <i>gdd_annual_ref</i> (mean annual cumulated GDD for the relevant <i>t_base</i>).
<i>pheno_fraction_ref_gdd5</i>	float	documentary	GDD5	The annual GDD5 against which <i>pheno_fraction</i> was originally calibrated (typical Mediterranean = 2200).
<i>pheno_fraction_ref_gdd10</i>	float	documentary	GDD10	As above, for <i>t_base</i> = 10°C references.
<i>pheno_bbch_start</i>	string	recommended	—	Human-readable BBCH stage at the start of the window.
<i>pheno_bbch_end</i>	string	recommended	—	Human-readable BBCH stage at the end of the window.

The engine selects the source of the window as follows:

1. If both *pheno_fraction* and *gdd_annual_ref* (computed internally from the weather series) are available → **use the normalised form**: [*lo* × *ref*, *hi* × *ref*].
2. Otherwise, if *phenology_window_gdd* is set → use the absolute form.
3. Otherwise → no gating, $\mu_{pheno} = 1$ for every day.

pheno_hi ≥ 9000 is treated as “always in season” (used for year-round tropical pests).

1.4 Pathogen biology

Field	Type	Status	Unit	Meaning
<i>min_streak</i>	int	recommended	days	Minimum number of consecutive favourable days needed to infect / establish. Most insects: 1. Wetness-dependent pathogens: 2 (Venturia, Plasmopara, Taphrina, Monilia).
<i>min_wetness_hours_critical</i>	float	optional	h/day	If <i>wetness_h</i> < <i>this value</i> , the day cannot reach a Critical score — it is capped just below <i>RISK_THRESHOLD_CRITICAL</i> . Applies the Mills/Goidanich logic.

<i>min_wetness_hours_high</i>	float	optional	h/day	Same as above, but caps below <i>RISK_THRESHOLD_HIGH</i> .
<i>min_free_water_hours</i>	float	optional	h/day	Documentary — currently not consumed by the engine. Reserved for sites where a film of liquid water is measured directly.
<i>wetness_effect</i>	string	optional	—	Free-text qualifier (e.g. “increases sporulation”). Documentary.
<i>rain_window_days</i>	int (1 or 3)	optional	days	Rainfall window length: 1 = raw daily, 3 (<i>default</i>) = 3-day rolling sum. Switch to 1 for pests with a 24-h infection rule (Plasmopara/Goidanich, fire blight).
<i>two_fifteens_rule</i>	string	optional	—	Compact citation of the “Two Fifteens” rule (Anarsia / Grapholita). Documentary.

1.5 Insect generations (documentary)

For lepidoptera and other multi-generation insects, additional fields preserve the published flight/oviposition GDD landmarks. They are **not yet consumed** by the engine but are kept in the JSON because (a) they are part of the scientific record, and (b) they are reserved for future treatment-window predictors.

Field	Type	Meaning
<i>gdd_first_flight</i>	int	GDD at first adult flight
<i>gdd_peak_first_gen</i>	int	GDD at peak first generation
<i>gdd_second_flight</i>	int	GDD at second flight
<i>gdd_first_generation</i>	int	GDD at first complete generation
<i>gdd_second_generation</i>	int	GDD at second generation
<i>gdd_third_generation</i>	int	GDD at third generation
<i>gdd_emergence_threshold</i>	int	GDD at first adult emergence
<i>gdd_peak_emergence</i>	int	GDD at the peak of adult emergence
<i>gdd_oviposition_end</i>	int	GDD at the end of oviposition

2. Validation rules and defaults

The loader applies a few defaults silently:

Field	If missing
<i>t_base</i>	defaults to 10.0
<i>pheno_t_base</i>	defaults to <i>t_base</i>

<i>min_streak</i>	defaults to 1
<i>rain_window_days</i>	defaults to 3
<i>phenology_window_gdd</i>	defaults to <i>[None, None]</i> → no gating
<i>pheno_fraction</i>	defaults to <i>[None, None]</i> → falls back to <i>phenology_window_gdd</i>
<i>t_lethal_max, t_lethal_min</i>	optional — guard not applied if missing
<i>min_wetness_hours_critical/high</i>	optional — Mills gate not applied if missing

Other gotchas:

1. **The pest_key must be identical** to the key extracted from the Excel column. The loader trims whitespace and collapses parenthetical variants (*Taphrina deformans (fruits)* is treated as one key), but typos or hidden characters silently break the match and the pest will fall back to the global defaults — see document 05 for the troubleshooting checklist.
 2. **_metadata entries are skipped**: any key whose value is not a dict is ignored. This makes it safe to add a top-level commentary block, e.g. { *"_metadata": { "last_review": "2026-04-15" }* }, *"Apple": { ... } }*.
 3. **String numbers are not coerced**: write *5.0*, not *"5.0"*.
 4. **JSON syntax**: trailing commas and *//* comments are *not* legal JSON. To keep comments, mirror the doc in a *.jsonc* file or use the *note* field.
-

04 — Climate criteria and disease selection

Which weather variables drive the model, how they enter each step of the inference, and how a set of diseases is chosen for a given crop. This document answers two complementary questions: (a) what the model “looks at” each day, and (b) how to decide which pests/pathogens should be modelled for a new crop.

1. Which climate variables the model uses

The PDM reads five raw daily variables from weather data: maximum and minimum temperature, relative humidity and rainfall. From these five inputs `compute_features()` derives ~30 columns. The ones that **actually feed the score** are:

Derived feature	What it represents	Used by
<i>temp_avg</i>	(Tmax + Tmin) / 2	Daily T for insects + lethal-T guard
<i>temp_avg_7d</i>	7-day rolling mean of <i>temp_avg</i>	T for pathogens
<i>humidity_7d</i>	7-day rolling mean of RH	RH for pathogens
<i>rain_3d</i>	3-day rolling sum of rainfall	Rainfall for most pests
<i>rainfall</i> (daily)	raw daily rainfall	Rainfall for pests with <i>rain_window_days: 1</i>
<i>wetness_h</i>	Estimated daily leaf-wetness hours	Mills-type gate
<i>gdd_cum_{X}b</i>	Cumulative GDD with base X °C	Phenological gating
<i>streak_hum60/70</i>	Consecutive days with RH ≥ 60 / 70 %	Streak penalty

2. How each variable enters the fuzzy decision

Every Excel rule mentions three bounds: humidity, temperature, rainfall threshold. The engine fuzzifies each of them against the day’s effective value with a trapezoidal membership function. The three memberships are then aggregated with a Mamdani **min** operator:

$$\mu_{rule} = \min(\mu_T, \mu_{RH}, \mu_{rain})$$

This is the **standard fuzzy AND**. Practically it means that the *weakest* of the three favourability factors dominates the rule’s activation. If RH and T are perfect but rainfall is far below the threshold, the rule will hardly fire which is biologically correct for foliar pathogens that *need* a leaching event to release spores.

2.1 Why pathogens use rolling means and insects do not

Insects react fast: a *Cydia* adult flies on a warm afternoon and lays eggs the same evening. Daily T captures this. Fungi instead require sustained humidity *and* a leaf-wetness window: a single rainy day does not infect *Venturia* unless followed by enough wetness hours. The 7-day rolling mean captures the slow build-up.

This dichotomy is wired into `score_day()` through the `_PATHOGEN_KEYWORDS` set (genera of fungi and bacteria that respond slowly). When a pest matches the keywords, the model substitutes *temp_avg_7d* and *humidity_7d* for the daily values; otherwise it uses the daily

values. There is one explicit exception: *Erwinia amylovora* because fire blight epidemics are episodic 1–3 day events, so daily T captures them better than a 7-day MA.

2.2 The leaf-wetness gate

Some fungi (Venturia, Botrytis, Monilia, Taphrina) only infect if the leaf surface remains wet for a minimum number of hours per day, at the right temperature. Mills (1944) for Venturia and Gottwald & Bertrand (1983) for Taphrina captured this with empirical tables.

In the PDM the wetness is **estimated** from RH and rainfall via two empirical lookups (*WETNESS_HOURS_BY_RH*, *WETNESS_HOURS_BY_RAIN* in *config.py*) and stored in the *wetness_h* column. When the pest declares *min_wetness_hours_critical* and/or *min_wetness_hours_high*, the score is **capped** to the appropriate level if *wetness_h* falls short. This is a *cap*, not a multiplier: the rule may still fire, but its severity cannot exceed the Mills threshold.

2.3 The streak penalty

Several pathogens need at least *N consecutive* days of high humidity to infect (Taphrina deformans: 2 days with ≥ 15 h wetness). The model counts consecutive days with $RH \geq 70\%$ (*streak_hum70* for pathogens) or $\geq 60\%$ (*streak_hum60* for insects). If the streak is below *min_streak*, the score is multiplied by $streak / min_streak$, with a soft floor of $STREAK_MIN_FACTOR = 0.30$ only when *min_streak* = 1. Hard floor (zero) applies when $min_streak \geq 2$.

3. From climate criteria to disease selection

Which diseases should I model for crop X? This is the central question when populating the database for a new crop. The answer comes from a triple intersection:

1. **Epidemiological relevance** — pests that are known to cause losses on that crop in the target geography (consult the EPPO global database, local extension services, regional IPM bulletins);
2. **Climatic compatibility** — pests whose environmental optimum overlaps the climate of the site. A pest like *Pseudocercospora fijiensis* (black sigatoka on banana) only matters in tropical climates;
3. **Phenological alignment** — pests whose susceptibility window falls inside the growing season of the crop in the target location.

3.1 Climatic screening worksheet

When evaluating whether a pest belongs in the database for a given crop/climate, walk through this six-question matrix and look up the answers in the literature. The output of this checklist is one JSON entry plus 4–6 Excel rules.

Question	Source	What to capture
What is the temperature optimum ?	EPPO datasheet, primary literature	<i>t_optimal_min</i> , <i>t_optimal_max</i>
What is the temperature limit ?	Lab studies, regional surveys	<i>t_lethal_max</i> , optionally <i>t_lethal_min</i>
Does the pathogen require leaf wetness ?	Mills tables, Goidanich, Gottwald-	<i>min_wetness_hours_critical/high</i> , $min_streak \geq 2$

How many hours?	Bertrand	
Does it require rainfall events ? Daily or cumulated?	Goidanich rule, infection biology	<i>rain_window_days</i> : 1 or 3, rule's <i>rain_min</i>
What BBCH stages of the host are susceptible?	Phytopathology manuals, regional bulletins	<i>pheno_bbch_start/end</i> , <i>phenology_window_gdd</i>
What is the typical GDD season in the target site?	Local climatology	<i>pheno_fraction_ref_gdd5</i> / <i>gdd10</i>

4. Climate issues

Three failure modes worth mentioning explicitly:

1. **Sentinel-bounded rules.** A rule such as “ $T < 5 \rightarrow \text{Low}$ ” appears in the Excel as $T < 5$, which the parser stores as $(-999, 5)$. Without *FUZZY_SPAN_CAP*, the ramp width would be 150 °C — the rule would fire at 25 °C. The cap protects against this.
2. **Missing weather days.** The loader interpolates temperature linearly but only forward-fills RH and rainfall (and sets a 70% default if both bracket NaNs). For long gaps (> 5 days) the model still runs but may produce smoothed-out scores. Always inspect the weather first if a result looks suspicious.
3. **First-year GDD instability.** *gdd_annual_ref* is approximated linearly until at least 350 days of data are available. For a first deployment, either provide a calibration year or be aware that the normalised phenological windows (*pheno_fraction*) will use a less-stable reference. After one full year the reference stabilises.

05 — Database population guide

Step-by-step procedure to extend the model with a new crop, a new pest, or a new rule. The same workflow applies whether you are starting from a blank database or adding to the existing 19-crop catalogue.

The PDM has **two data files** that must remain mutually consistent:

File	Holds	Granularity
<i>db_crops_PDM.xlsx</i>	Fuzzy rules (one row = one risk band)	≈ 4–6 rows per (<i>crop</i> , <i>pest</i>)
<i>pest.json</i>	Biological parameters	1 entry per (<i>crop</i> , <i>pest</i>)

The matching key is the pair (*crop*, *pest_key*). Mismatches between the two files are the single most frequent source of bugs.

1. Literature review

Before opening either file, capture the following from primary literature, EPPO datasheets, and regional IPM bulletins:

Item	Used for	Typical source
Common name	<i>label</i>	EPPO global database
Temperature optimum	<i>t_optimal_min/max</i>	Primary epidemiology paper
Survival range	<i>t_lethal_min/max</i>	Lab thermal-tolerance studies
GDD base temperature for development	<i>t_base</i>	Insect development models
Phenology window (BBCH stages)	<i>pheno_bbch_*</i> , <i>phenology_window_gdd</i>	Pathology manuals, growers' guides
Leaf wetness requirement	<i>min_wetness_hours_*</i> , <i>min_streak</i>	Mills (1944), Gottwald-Bertrand (1983), etc.
Rainfall infection rule	<i>rule_rain_min</i> , <i>rain_window_days</i>	Goidanich (1975), Maryblyt / Cougarblight
Bibliography	<i>source</i>	DOI of the two or three most cited references

Capture all references *inside* the JSON entry (*source*, *model_reference*, *note*, *mills_table_reference*). The audit trail must live in the data, not in a side document.

2. Define the pest_key

The *pest_key* is the column the engine joins on. The rules in *risk_model.py* are:

1. The text in the “Pest/Pathogen” column of the Excel file is split at the **first comma**: everything before the comma is the candidate *pest_key*, everything after is treated as a friendly description.
2. The same string must appear verbatim as a key inside *pest.json* under the matching crop.

Convention: use the **scientific name** (binomial Latin, capitalised genus) as the pest_key. Avoid abbreviations, plural forms, asterisks, footnote markers.

3. Edit *pest.json*

Add the new pest under the right crop. If the crop does not exist, create it.

3.1 Minimal entry

```
"Strawberry": {
  "Botrytis cinerea": {
    "label": "Grey mould",
    "t_base": 5.0,
    "t_optimal_min": 15.0,
    "t_optimal_max": 22.0,
    "t_lethal_max": 32.0,
    "min_streak": 2,
    "min_wetness_hours_critical": 15,
    "min_wetness_hours_high": 10,
    "phenology_window_gdd": [80, 1000],
    "pheno_fraction": [0.036, 0.455],
    "pheno_fraction_ref_gdd5": 2200,
    "pheno_bbch_start": "BBCH 60 (Flowering)",
    "pheno_bbch_end": "BBCH 89 (Harvest)",
    "source": "Williamson et al. (2007) Mol. Plant Pathol. 8:561-580;
doi:10.1111/j.1364-3703.2007.00417.x"
  }
}
```

3.2 Recommended fields per pest type

Pest type	Always set	Often set	Avoid
Insect with thermal-driven cycle (Cydia, Anarsia)	<i>t_base</i> , <i>phenology_window_gdd</i> or <i>pheno_fraction</i> , <i>t_optimal_*</i> , <i>t_lethal_*</i>	<i>gdd_first_flight</i> , <i>gdd_peak_*</i> (documentary)	<i>min_wetness_hours_*</i>
Fungus with leaf-wetness requirement (Venturia, Botrytis, Plasmopara, Taphrina, Monilia)	All thermal fields, <i>min_streak</i> ≥ 2, <i>min_wetness_hours_*</i>	<i>mills_table_reference</i> , sometimes <i>rain_window_days: 1</i>	—
Bacterium with single thermal threshold (Erwinia)	<i>t_base</i> = bacterial threshold, <i>pheno_t_base</i> = host GDD base	<i>t_lethal_max</i> , <i>model_reference</i>	<i>min_streak</i> ≥ 2 (epidemic, not sustained)
Mite or sucking insect (Myzus, Eutetranychus)	<i>t_base</i> , <i>t_optimal_*</i> , <i>t_lethal_*</i>	<i>min_streak: 1</i>	leaf-wetness gates

4. Edit *db_crops_PDM.xlsx*

Open the worksheet in Excel. Each row is one rule. Add 4–6 rows for the new pest. A canonical set looks like:

Crop	Pest / Pathogen	Humidity (%)	Temperature (°C)	Min. Rainfall (mm)	Risk Level	Basis
Strawberry	Botrytis cinerea, Grey mould	>90%	15<=T<=22	3 mm	Critical	Fact
Strawberry	Botrytis cinerea, Grey mould	>80%	12<=T<=25	1 mm	High	Fact
Strawberry	Botrytis cinerea, Grey mould	>70%	10<=T<=27	whatever	Moderate	Fact
Strawberry	Botrytis cinerea, Grey mould	<60%	whatever	whatever	Low	Fact
Strawberry	Botrytis cinerea, Grey mould	whatever	T<5	whatever	Low	Fact
Strawberry	Botrytis cinerea, Grey mould	whatever	T>32	whatever	Low	Fact

4.1 Column reference

Column	Required format	Notes
<i>Crop</i>	text	Must match the JSON crop key exactly
<i>Pest / Pathogen</i>	<i>Genus species, common-name (possibly +variant)</i>	Drives <i>pest_key</i> extraction
<i>Humidity (%)</i>	<i>60-80%, >=70%, <60%, whatever</i>	Parsed into (<i>lo</i> , <i>hi</i>)
<i>Temperature (°C)</i>	<i>10<=T<=20, T>30, T<5, whatever</i>	Parsed into (<i>lo</i> , <i>hi</i>)
<i>Min. Rainfall (mm)</i>	<i>5 mm, >=3 mm, whatever</i>	Parsed into <i>rain_min</i> (0 if whatever)
<i>Risk Level</i>	<i>Critical, High, Moderate, Low</i>	Maps to risk scores 100 / 80 / 40 / 10
<i>Basis</i>	<i>Fact</i>	Free-text label; currently informational only

4.2 Rule-design tips

- Always include at least one **Critical** rule (the optimum) and at least two **Low** rules (cold and hot lethal cases). The Low rules act as sanity boundaries even when the lethal-T guard already exists in the JSON.
- The fuzzy aggregator picks the highest weighted defuzzification, so partially-overlapping bands are *fine*: they smooth the score around the boundaries.

- Use *whatever* for any factor that is biologically irrelevant for that particular band. The model maps it to $(0, 100) / (-999, 999)$, which contributes $\mu = 1$ — neutral on the AND aggregation.
- Always keep the **pest_key identical** to the JSON pest_key. Pay attention to capitalisation, hyphens and trailing spaces.

5. Update *config.py*

Situation	Edit <i>config.py</i> ?	Field
New pest has a <i>t_base</i> value already in <i>PEST_T_BASES</i>	No	—
New pest has a <i>t_base</i> value not yet in <i>PEST_T_BASES</i>	Yes	Add it to <i>PEST_T_BASES</i> , e.g. $\{0.0, 4.0, 5.0, \dots, 18.0, 22.0\}$
Changed risk thresholds for a local agronomic context	Yes	<i>RISK_THRESHOLD_*</i>
Calibrated weather station with leaf-wetness sensor	Yes	<i>WETNESS_HOURS_BY_RH/RAIN</i>
Site located in southern hemisphere	Yes	<i>GDD_RESET_MONTH = 7</i>

If you do not update *PEST_T_BASES* when introducing a new *t_base*, the model falls back to the 5°C cumulative GDD column for that pest — biologically incorrect for, say, a pest with *t_base* = 22°C.

6. Run and validate

6.1 Smoke test

```
python main.py --crop Strawberry --validate
```

Look at the validation summary printed at the end:

Crop	Pest (short)	Crit	High	Mod	Low
Strawberry	Grey mould	12	34	87	245

Expected pattern for a sound entry: a small-but-non-zero count of Critical, a larger High, and progressively more Moderate / Low. Anomalies to investigate:

Symptom	Likely cause
0 events in every class except Low	pest_key mismatch — JSON entry not picked up; lethal-T guard too strict
All events Critical	Rules too lenient; thresholds too low
100% “Out of season”	<i>phenology_window_gdd</i> empty or <i>pheno_fraction</i> window outside the weather series
“Out of season” oscillates around the start of the window	OK if the soft margin is doing its job

6.2 Spot-check a single day in the CSV

Open *output/risk_results.csv*, filter by *crop = Strawberry*, *pest = Botrytis cinerea*, *Grey mould*, pick a day with score ≈ 90 , read the *detail* column. It should look like:

```
best_rule=Critical mu=0.93 mu_pheno=1.00 streak=5d wh=18h
```

This single line tells you:

- the **dominant rule** is the Critical band, with very high μ (the day is genuinely on the rule's plateau);
- the host is fully in season;
- 5 consecutive days of high RH;
- 18 hours of estimated leaf wetness, well above the *min_wetness_hours_critical* = 15.

If any of these numbers contradicts your expectation, the corresponding rule or JSON field is suspect.

6.3 Re-generate the PDF

```
python main.py --crop Strawberry
```

Open *output/pest_risk_report.pdf*. The risk-timeline page must show a curve that respects seasonality (no Critical days in January for a temperate fungus) and the classification colour bands must align with the score thresholds.

7. Common failure modes and recovery

Failure	Diagnosis	Fix
<i>ValueError: No rules found for crop='X'</i>	Crop name typo in CLI argument or Excel	Check <i>python main.py --all</i> shows the crop in the “Available crops” list
Pest receives only Low scores all year	pest_key mismatch — JSON entry not loaded, defaults applied	Print <i>pest_config.keys()</i> from a Python REPL and confirm the tuple is present
Score never reaches Critical even on optimum days	Mills wetness gate set too high, or no rule with Risk Level = Critical	Inspect <i>wetness_h</i> in CSV; relax <i>min_wetness_hours_critical</i> or add a Critical rule
GDD curve flat at zero	<i>t_base</i> is too high for the climate (e.g. 22 °C in a temperate site)	Verify <i>t_base</i> against the literature; check <i>gdd_cum_{X}b</i> column directly
Phenology window collapses to 0 width	Both <i>pheno_fraction[0]</i> and <i>[1]</i> written the same	Fix the second value (open / closing GDD fractions are different)
<i>KeyError: 'gdd_cum_22b'</i>	<i>t_base</i> = 22 not added to <i>PEST_T_BASES</i>	Add the value to <i>PEST_T_BASES</i> in <i>config.py</i>
JSON parse error on load	Stray comma / unescaped quote / // comment	Validate the file at https://jsonlint.com before retrying